

Rapport Final

Application de Gestion de Bibliotheque Universitaire

Universite Norbert Zongo (UNZ)

Cours : Genie Logiciel — Methode Agile Scrum

Mai 2026

Depot GitHub :

<https://github.com/moumounibanao0-afk/bibliotheque-univ-nz1>

Table des Matieres

1. Introduction	3
2. Analyse des Exigences	4
3. Modelisation et Conception	6
4. Architecture et Design Patterns	8
5. Implementation et Tests	10
6. Gestion Agile (Scrum)	12
7. Difficultes rencontrees et Lecons apprises	14
8. Conclusion	15

1. Introduction

1.1 Contexte du projet

L'Université Norbert Zongo (UNZ), établissement d'enseignement supérieur au Burkina Faso, fait face à des défis dans la gestion de sa bibliothèque universitaire. Le système actuel, largement manuel, présente des limitations importantes en termes d'efficacité, de traçabilité et d'accessibilité pour les étudiants et le personnel.

Dans le cadre de sa politique de modernisation et de transformation numérique, l'UNZ a lancé ce projet de développement d'une application web complète de gestion de bibliothèque. Cette initiative s'inscrit également dans le cadre pédagogique du cours de Génie Logiciel, permettant aux étudiants d'appliquer les concepts théoriques à un cas réel d'envergure.

1.2 Objectifs du projet

Ce projet vise plusieurs objectifs complémentaires :

- **Objectif fonctionnel** : Développer une application web professionnelle permettant la gestion complète d'une bibliothèque universitaire (catalogue, emprunts, réservations, utilisateurs, statistiques).
- **Objectif pédagogique** : Appliquer l'ensemble des concepts du Génie Logiciel vus en cours : méthodologie Agile Scrum, patterns de conception, principes SOLID, tests unitaires, modélisation UML, déploiement cloud.
- **Objectif technique** : Maîtriser les technologies modernes de développement web (Spring Boot, MySQL, JWT, Docker, API REST) et les outils collaboratifs (Git, GitHub Projects).
- **Objectif qualité** : Produire un code maintenable, bien testé (couverture $\geq 65\%$), documenté et versionné selon les bonnes pratiques de l'industrie.

1.3 Périmètre fonctionnel

L'application couvre les fonctionnalités suivantes :

- Gestion complète du catalogue avec catégories et rayons
- Recherche avancée multi-critères (titre, auteur, ISBN, catégorie, mots-clés)
- Emprunt direct par les étudiants depuis l'interface web
- Réservation d'ouvrages indisponibles avec notification de disponibilité
- Gestion des retours avec calcul automatique des pénalités en FCFA
- Système de notification automatique par email (confirmation, rappels, alertes)
- Historique complet des emprunts et réservations par étudiant
- Gestion des utilisateurs avec 3 rôles distincts et authentification sécurisée JWT
- Tableau de bord avec statistiques et graphiques pour l'administration
- Génération de rapports sur les retards et les ouvrages les plus empruntés

1.4 Organisation du rapport

Ce rapport présente le travail réalisé durant les 3 sprints de développement. Il est structuré en 8 chapitres couvrant l'analyse, la conception, l'implémentation, les tests et la gestion de projet selon la méthodologie Scrum.

2. Analyse des Exigences

2.1 Identification des acteurs

Le systeme s'articule autour de 3 acteurs principaux, chacun ayant des besoins et des droits d'accès spécifiques :

Acteur	Description	Responsabilites principales
Etudiant	Utilisateur principal du systeme, membre de la communauté universitaire	Consultation du catalogue, recherche d'ouvrages, emprunt direct, réservation
Bibliothecaire	Personnel de la bibliotheque charge de la gestion du catalogue	Gestion du catalogue (ajout/modification/suppression d'ouvrages), gestion des emprunts
Administrateur	Responsable technique et fonctionnel du systeme	Gestion des comptes utilisateurs (3 roles), configuration du systeme

2.2 Exigences fonctionnelles detaillees

Les exigences fonctionnelles ont ete identifiees et priorisees selon la methode MoSCoW :

ID	Exigence	Priorite	Complexite
EF01	Gestion du catalogue avec categories et rayons	Must have	Moyenne
EF02	Recherche avancee multi-criteres	Must have	Moyenne
EF03	Emprunt direct par l'etudiant avec verification disponibilite	Must have	Haute
EF04	Reservation quand tous exemplaires empruntes	Should have	Moyenne
EF05	Calcul automatique penalites selon type etudiant	Should have	Moyenne
EF06	Notification email automatique (7 types)	Could have	Haute
EF07	Historique complet emprunts/reservations	Must have	Basse
EF08	Authentification JWT avec 3 roles	Must have	Haute
EF09	Statistiques et graphiques temps reel	Should have	Moyenne
EF10	Rapports de retard et penalites	Could have	Basse

2.3 Exigences non fonctionnelles

Les exigences non fonctionnelles garantissent la qualite globale du systeme :

Categorie	Exigence	Metrique	Resultat obtenu
Securite	Authentification JWT + protection HTTP endpoints proteges	100% endpoints proteges	■ JWT 24h + roles
Performance	Temps de reponse < 2s	95% requetes < 2s	■ Moyenne 0.8s
Disponibilite	Uptime >= 99%	Monitoring 24/7	■■ Depends Render
Maintenabilite	Code respectant SOLID + 4 patterns	Audit qualite	■ 4/4 patterns
Testabilite	Couverture tests >= 65%	JaCoCo report	■ 74.8%
Portabilite	Interface responsive mobile/desktop	Tests multi-devices	■ Bootstrap responsive
Utilisabilite	Interface intuitive sans formation	Tests utilisateurs	■ 3 interfaces distinctes

2.4 User Stories completes

Les 15 User Stories ont ete redigees selon le format standard Scrum : En tant que [role], je veux [action], afin de [benefice].

ID	User Story complete	Criteres d'acceptation	Sprint
US01	En tant qu'etudiant, je veux m'inscrire avec mon email	Un email valide au format @.fr	Sprint 1
US02	En tant qu'utilisateur, je veux me connecter	Authentification par mot de passe	Sprint 1
US03	En tant que bibliothecaire, je veux ajouter/modifier une valise de livres	Un livre valide, une valise valide	Sprint 1
US04	En tant qu'etudiant, je veux consulter le catalogue	Affichage des livres disponibles	Sprint 1
US05	En tant qu'etudiant, je veux rechercher un livre	Recherche par mots-clés	Sprint 1
US06	En tant qu'etudiant, je veux emprunter directement	Emprunt direct, durée de prêt	Sprint 2
US07	En tant que bibliothecaire, je veux enregistrer	Calcul automatique de la date de retour	Sprint 2
US08	En tant qu'etudiant, je veux réserver un ouvrage	Reservation, notification de disponibilité	Sprint 2
US09	En tant que bibliothecaire, je veux voir les retards	Tableau des retards, calcul des amendes	Sprint 2
US10	En tant qu'etudiant, je veux consulter mon historique	Historique des emprunts, réservations	Sprint 2
US11	En tant qu'etudiant, je veux recevoir des emails	Notifications par email	Sprint 2
US12	En tant qu'administrateur, je veux gérer les utilisateurs	Gestion des utilisateurs, rôles	Sprint 3
US13	En tant qu'administrateur, je veux voir les statistiques	Tableaux de bord, graphiques	Sprint 3
US14	En tant que bibliothecaire, je veux gérer les catégories	Gestion des catégories, attributions	Sprint 3
US15	En tant qu'administrateur, je veux générer des rapports	Génération de rapports, export	Sprint 3

3. Modelisation et Conception

3.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation modelise les interactions entre les acteurs et le systeme. Il identifie 18 cas d'utilisation repartis selon les acteurs :

Etudiant (9 cas) : S'inscrire, Se connecter, Consulter catalogue, Rechercher ouvrage, Emprunter ouvrage, Reserver ouvrage, Annuler reservation, Consulter historique, Recevoir notifications

Bibliothecaire (7 cas) : Se connecter, Ajouter ouvrage, Modifier ouvrage, Supprimer ouvrage, Enregistrer retour, Voir retards, Gerer categories

Administrateur (5 cas) : Se connecter, Creer utilisateur, Modifier utilisateur, Supprimer utilisateur, Voir statistiques, Generer rapports

Les relations entre cas d'utilisation incluent des <> (Emprunter inclut Verifier disponibilite) et des <> (Retourner peut etendre vers Calculer penalite).

3.2 Diagramme de classes detaille

Le modele de donnees suit une architecture orientee objet avec heritage et associations :

Classe	Attributs (selection)	Methodes principales	Relations
Utilisateur	id:Long, nom:String, prenom:String, email:String, motDePasse:String, role:Role, statut:StatutUtilisateur	getFullString(), isAdmin(), isBibliothecaire(), isEtudiant(), isBoursier(), isRetard(), isEmprunteur(), isReserveur(), isHistorique(), isNotifications()	1-N vers Emprunt, 1-N vers Reservation
Etudiant extends Utilisateur	matricule:String (INE), filiere:String, boursier:Boolean	getMatricule(), isBoursier()	1-N vers Emprunt, 1-N vers Reservation
Bibliothecaire extends Utilisateur	matricule:String, service:String	getMatricule()	Gere les operations
Administrateur extends Utilisateur	id:Long, nom:String	getSuperAdmin()	Gere les utilisateurs
Ouvrage	id:Long, titre:String, auteur:String, isbn:String, categorie:Categorie, statut:StatutOuvrage	getFullString(), isDisponible(), isEmprunteur(), isReserveur(), isHistorique(), isNotifications()	1-N vers Emprunt, 1-N vers Reservation
Categorie	id:Long, nom:String, description:String	getFullString()	1-N vers Ouvrage
Emprunt	id:Long, dateEmprunt:LocalDate, dateRetour:LocalDate, statut:StatutEmprunt	getFullString(), isDisponible(), isEmprunteur(), isReserveur(), isHistorique(), isNotifications()	1-N vers Emprunt, 1-N vers Reservation
Reservation	id:Long, dateReservation:LocalDate, statut:StatutReservation	getFullString(), isDisponible(), isEmprunteur(), isReserveur(), isHistorique(), isNotifications()	N-1 vers Etudiant, N-1 vers Ouvrage

Enumerations : StatutEmprunt {EN_COURS, EN_RETARD, RETOURNE}, StatutReservation {EN_ATTENTE, CONFIRMEE, ANNULEE}, Role {ETUDIANT, BIBLIOTHECAIRE, ADMINISTRATEUR}

3.3 Diagrammes de sequence detailles

3.3.1 Sequence : Connexion utilisateur

1. L'utilisateur saisit email et mot de passe sur login.html
2. Le frontend valide le format email (@unz.bf) et envoie POST /api/auth/login
3. AuthController intercepte la requete et extrait les credentials
4. UtilisateurRepository.findByEmail() recherche en base MySQL
5. Si trouve, PasswordEncoder.matches() compare les mots de passe (BCrypt)
6. JwtUtil.genererToken() cree un JWT valide 24h avec email et role
7. Le controller retourne {token, role, nom, prenom, id}
8. Le frontend stocke le token en localStorage et redirige selon le role

3.3.2 Sequence : Emprunt par etudiant

1. L'etudiant clique sur le bouton 'Emprunter' dans le catalogue
2. Une confirmation modale s'affiche avec les details (titre, duree 14j)
3. Apres confirmation, POST /api/emprunts avec {etudiantId, ouvrageId}
4. Spring Security valide le token JWT et verifie le role ETUDIANT
5. EmpruntService.creerEmprunt() verifie ouvrage.exemplairesDisponibles > 0
6. Creation emprunt avec dateRetourPrevue = aujourd'hui + 14 jours
7. Ouvrage.exemplairesDisponibles decremente de 1
8. NotificationService.envoyerConfirmationEmprunt() envoie email via SMTP Gmail
9. Frontend affiche message succes et rafraichit le catalogue

4. Architecture et Design Patterns

4.1 Architecture 4+1 vues (Kruchten)

Vue Logique : Architecture en couches :

- Couche Presentation : HTML5/CSS3/JavaScript. 3 interfaces (login.html, admin.html, etudiant.html). Responsive Bootstrap. Communication AJAX avec backend.
- Couche Application : Controllers REST Spring Boot. Gestion des requetes HTTP, validation, transformation JSON. 8 controllers principaux.
- Couche Domaine : Services metier avec logique. 6 services (OuvrageService, EmpruntService, ReservationService, NotificationService, etc.). Implementation des Design Patterns.
- Couche Infrastructure : Repositories JPA + MySQL. Acces base de donnees via Spring Data JPA. Transactions geres automatiquement.

Vue Developpement : Organisation des packages Java :

Package	Contenu	Responsabilite
controller/	8 classes @RestController	API REST, routage HTTP, validation entrees
service/	6 classes @Service	Logique metier, transactions, orchestration
repository/	7 interfaces @Repository	Acces donnees JPA, requetes SQL automatiques
model/	8 entites JPA	Modele de donnees, mapping ORM
config/	4 classes @Configuration	Configuration Spring Security, JWT, CORS, SMTP
strategy/	2 classes + interface	Pattern Strategy pour calcul penalites
factory/	1 classe	Pattern Factory pour creation utilisateurs
observer/	3 classes + interfaces	Pattern Observer pour notifications

Vue Processus : Flux d'execution type :

Requete HTTP → Filtre CORS → Spring Security (JWT) → Controller (routing) → Service (logique + patterns) → Repository (JPA) → MySQL → Transaction commit → Reponse JSON

Vue Physique : Deploiement :

Navigateur Web → Internet → Render.com (Docker Container Spring Boot port 8081) → Internet → FreeSQLDatabase.com (MySQL port 3306)

Vue Scenarios : 15 User Stories couvrant 3 sprints de 2 semaines.

4.2 Principes SOLID appliqués

Principe	Description	Application concrète dans le projet
S — Single Responsibility	Une classe = une responsabilité	OuvrageService gère uniquement les ouvrages (CRUD, recherche). EmpruntService gère uniquement les emprunts.
O — Open/Closed	Ouvert extension, fermé modification	Interface CalculPenalite permet d'ajouter de nouvelles stratégies de pénalités.
L — Liskov Substitution	Sous-classes substituables	Etudiant, Bibliothécaire, Administrateur héritent de Utilisateur. Toute méthode d'Utilisateur peut être appelée sur ces sous-classes.
I — Interface Segregation	Interfaces spécifiques, pas génériques	Observable et Observateur sont petites et ciblées. CalculPenalite a une interface spécifique.
D — Dependency Inversion	Dépendre des abstractions	Controllers dépendent de interfaces Service (pas d'implementations). Services dépendent de interfaces Repository.

4.3 Design Patterns implémentés

Pattern 1 : Singleton (Création)

Classe : DatabaseConfig (@Configuration Spring)

Problème : Éviter la création de multiples instances de configuration de base de données, ce qui entraînerait des fuites mémoire et des connexions multiples inutiles.

Solution : Spring IoC Container gère DatabaseConfig comme Singleton. Une seule instance créée au démarrage de l'application et réutilisée partout.

Avantage : Optimisation ressources, cohérence configuration, pas de duplication.

Pattern 2 : Observer (Comportement)

Classes : Observable (interface), Observateur (interface), OuvrageDisponibleObservable (implémentation), NotificationService (observateur)

Problème : Notifier automatiquement les étudiants ayant réservé un ouvrage quand celui-ci devient disponible après un retour.

Solution : Lors d'un retour, EmpruntService.retournerEmprunt() appelle observable.notifierObservateurs(). Tous les observateurs inscrits (étudiants avec réservation active) reçoivent une notification email automatique.

Avantage : Découplage complet entre système d'emprunt et système de notification. Ajout de nouveaux observateurs sans modifier le code existant.

Pattern 3 : Factory (Création)

Classe : UtilisateurFactory

Problème : Créer le bon type d'utilisateur (Etudiant, Bibliothécaire, Administrateur) sans exposer la logique de création au contrôleur.

Solution : UtilisateurFactory.creerUtilisateur(role, données) retourne l'instance correcte selon le rôle. Centralisation de la logique de création complexe.

Avantage : Facilite l'ajout de nouveaux types utilisateurs. Code contrôleur reste simple. Logique création isolée.

Pattern 4 : Strategy (Comportement)

Interface : CalculPenalite. Implémentations : PenaliteStandard (100 FCFA/jour), PenaliteBoursier (50 FCFA/jour)

Probleme : Le calcul des penalites varie selon le type d'etudiant. Coder des if/else dans EmpruntService violerait Open/Closed.

Solution : Chaque strategie implemente CalculPenalite.calculer(joursRetard). EmpruntService recoit une strategie par injection et l'utilise sans connaitre l'implementation.

Avantage : Changement d'algorithme sans modification du service. Ajout de nouvelles strategies (PenaliteDoctorant, PenaliteProfesseur) sans toucher au code existant. Respecte Open/Closed.

5. Implementation et Tests

5.1 Stack technique complete

Composant	Technologie	Version	Justification du choix
Langage	Java	21 LTS	Langage type fort, ecosysteme riche, performances, maintien long terme
Framework Backend	Spring Boot	3.2.0	Standard industrie, configuration automatique, ecosysteme complet (Security, ORM, Cloud, etc.)
ORM	Hibernate / JPA	6.3	Standard Java persistence, mapping objet-relationnel automatique, requetes complexes
Base de donnees	MySQL	8.4	SGBD relationnel robuste, gratuit, compatible JPA, performances excellentes
Securite	Spring Security + JWT	6.2	Authentification stateless, tokens securises, protection CSRF, autorisation
Frontend	HTML5 / CSS3 / Vanilla JS		Zero dependance, leger, compatible tous navigateurs, responsive natif
Framework CSS	Bootstrap	5.3	Composants pre-faits, responsive grid system, compatibilite mobile/desktop
Build Tool	Maven	3.9	Gestion dependances, builds reproductibles, plugins Spring Boot integres
Tests Unitaires	JUnit 5 + Mockito	5.10	Standard Java testing, mocks pour isolation, annotations claires
Couverture Code	JaCoCo	0.8.11	Rapports HTML/XML, integration Maven, seuils configurables
Conteneurisation	Docker	24	Portabilite, environnement reproductible, deploiement cloud facile
Deploiement	Render.com	—	Plan gratuit, deploiement Git auto, logs temps reel, SSL gratuit
Base de donnees Cloud	CloudSQLDatabase	—	MySQL gratuit 5MB, suffisant MVP, connexion SSL
Email SMTP	Gmail SMTP	—	Gratuit 500 emails/jour, fiable, configuration simple Spring Mail
Versionnement	Git + GitHub	—	Commits traces, branches, PR, GitHub Projects integre

5.2 Structure du code source

Le projet suit une architecture modulaire standard Spring Boot :

src/main/java/com/bibliotheque/ : Code source principal (33 classes)

src/main/resources/ : Fichiers configuration (application.properties, HTML/CSS/JS)

src/test/java/ : Tests unitaires (8 classes de test, 47 tests)

pom.xml : Gestion dependances Maven

Dockerfile : Configuration conteneur Docker

5.3 Tests unitaires details

Classe de test	Methodes testees	Nombre tests	Techniques utilisees	Couverture
OuvrageServiceTest	CRUD complet, recherche basique	5	Mocks Repository, assertions	85%
OuvrageServiceAvanceeTest	Recherche avancee multi-criteres	7	Tests parametrees, cas limites	90%
EmpruntServiceTest	Creation, retour, calcul penale	6	Mocks multiples, verification	80%
ReservationServiceTest	Creation, annulation, confirmation	5	Spy pour verifier interactions	75%
NotificationServiceTest	Envoi emails 7 types	4	Mock JavaMailSender, capture arguments	70%
ModelTest	Validation contraintes JPA	7	Validator javax, tests entites	95%
CategorieTest	CRUD categories	6	Integration H2 memoire	85%
FactoryTest	Creation utilisateurs par role	4	Verification types retournes	100%

AppTest	Chargement contexte Spring1	@SpringBootTest	100%
---------	-----------------------------	-----------------	------

5.4 Resultats de couverture (JaCoCo)

Package	Classes	Methodes	Lignes	Branches	Couverture
controller/	8/8	45/52	180/210	30/45	82%
service/	6/6	38/42	250/295	55/70	85%
model/	8/8	56/60	120/140	—	86%
config/	4/4	12/15	45/60	5/10	70%
strategy/	2/2	4/4	12/15	2/4	80%
factory/	1/1	3/3	8/10	—	80%
observer/	3/3	6/8	15/20	—	75%
TOTAL	33/33	164/184	630/750	92/129	74.8%

Analyse : Objectif 65% largement depasse (+9.8 points). Les packages core (service, controller) ont une excellente couverture (>80%). Les packages config et observer sont moins couverts car contiennent du code Spring auto-configuré difficile à tester unitairement.

5.5 Outils de qualite de code

- Maven Checkstyle : verification style code Java
- SpotBugs : detection bugs potentiels
- SonarLint : analyse qualite temps reel dans IDE
- Git pre-commit hooks : validation avant commit

6. Gestion Agile (Scrum)

6.1 Organisation de l'equipe

L'equipe a adopte la methodologie Scrum avec les roles suivants :

- **Product Owner** : Represente le client (Universite Norbert Zongo). Priorise le Product Backlog. Valide les User Stories.
- **Scrum Master** : Facilite les ceremonies Scrum. Supprime les obstacles. Veille au respect de la methodologie.
- **Developpeurs** : Equipe auto-organisee. Estimation des User Stories. Implementation et tests.

6.2 Sprints realises

Sprint 1 — Fondations et Catalogue (2 semaines)

Objectif : Mettre en place l'infrastructure technique et les fonctionnalites de base.

User Stories completees : US01 (Inscription), US02 (Connexion JWT), US03 (Gestion ouvrages), US04 (Catalogue), US05 (Recherche avancee)

Livrables techniques : Spring Boot configure, MySQL operationnel, JWT implemente, 3 interfaces HTML, 15 tests ecrits

Velocity : 20 points de complexite

Retrospective : Points positifs - Configuration rapide Spring Boot. Points d'amelioration - Manque documentation API initiale.

Sprint 2 — Emprunts et Reservations (2 semaines)

Objectif : Implementer le coeur fonctionnel metier.

User Stories completees : US06 (Emprunt direct), US07 (Retour), US08 (Reservation), US09 (Penalites), US10 (Historique)

Livrables techniques : Pattern Strategy penalites, Pattern Observer notifications, 18 tests supplementaires, Interface etudiant complete

Velocity : 18 points de complexite

Retrospective : Points positifs - Design Patterns bien integres. Points d'amelioration - Gestion des contraintes de clefs etrangeres difficile.

Sprint 3 — Notifications et Statistiques (2 semaines)

Objectif : Finaliser les fonctionnalites avancees et le deploiement.

User Stories completees : US11 (Emails), US12 (Gestion utilisateurs), US13 (Statistiques), US14 (Categories), US15 (Rapports)

Livrables techniques : SMTP Gmail configure, Dashboard graphiques, Deploiement Render, Documentation complete, 14 tests supplementaires

Velocity : 17 points de complexite

Retrospective : Points positifs - Deploiement cloud reussi. Points d'amelioration - Gestion des variables d'environnement complexe.

6.3 GitHub Projects

URL du projet : <https://github.com/users/moumounibanao0-afk/projects/12>

Le Product Backlog contient : 15 User Stories (US01-US15), 9 Daily Scrum (3 par sprint), 3 Sprint Review, 3 Sprint Retrospective

Colonnes Kanban : Todo (avant sprint), In Progress (durant sprint), Done (apres sprint review)

Etat final : 14 User Stories en Done, 1 User Story en In Progress (US15 Rapports - fonctionnel mais perfectible)

6.4 Ceremonies Scrum

Ceremonie	Frequence	Participants	Duree	Objectif
Sprint Planning	Debut de sprint	Toute l'equipe	2h	Selection US du backlog, estimation, repartition tâches
Daily Scrum	Quotidien	Developpeurs + SM	15min	Point avancement, obstacles, synchronisation
Sprint Review	Fin de sprint	Equipe + PO	1h	Demo US completees, feedback, validation
Sprint Retrospective	Fin de sprint	Toute l'equipe	1h	Bilan sprint, ameliorations continues
Backlog Refinement	Mi-sprint	PO + developpeurs	1h	Clarification US futures, split si necessaire

7. Difficultes rencontrees et Lecons apprises

7.1 Difficultes techniques majeures

Difficulte 1 : Configuration Spring Security avec JWT

Probleme : Spring Security 6.x bloquait toutes les requetes API par default, meme apres ajout du JwtFilter.

Symptome : 403 Forbidden sur tous les endpoints, y compris /api/auth/login.

Solution : Configuration SecurityFilterChain precise avec authorizeHttpRequests() et regles par role. Ajout de .permitAll() sur /api/auth/**. Ordre des filtres crucial : CORS avant JWT.

Lecon : Bien comprendre le cycle de vie des filtres Spring. Tester la securite incrementalement endpoint par endpoint.

Difficulte 2 : Mapping JPA avec categorield

Probleme : Le frontend envoyait {categorield: 3} mais le controller attendait un objet Categorie complet.

Symptome : Ouvrages crees sans categorie (categorie_id NULL en base).

Solution : Modification du controller pour accepter Map, extraction du categorield, puis CategorieRepository.findById() pour lier l'entite.

Lecon : Attention au mapping JSON <-> JPA. Les DTOs auraient simplifie ce probleme. Dans un projet reel, utiliser @JsonManagedReference/@JsonBackReference.

Difficulte 3 : Deploiement Render sans MySQL gratuit

Probleme : Render.com ne propose pas MySQL gratuit, seulement PostgreSQL. Migration couteuse car requetes specifiques MySQL.

Symptome : Application deployee mais erreur connexion base.

Solution : Utilisation FreeSQLDatabase.com pour MySQL gratuit externe. Configuration variables environnement SPRING_DATASOURCE_URL/USERNAME/PASSWORD.

Lecon : Anticiper les contraintes des plateformes cloud. Avoir un plan B (Railway, Clever Cloud). Tester localement avec variables d'environnement avant deploiement.

Difficulte 4 : Conflits de ports en developpement

Probleme : Port 8081 deja utilise lors des redemarrages, causant des echecs de lancement.

Symptome : BindException: Address already in use.

Solution : Commande fuser -k 8081/tcp avant chaque lancement. Creation d'un script shell de demarrage automatique.

Lecon : Automatiser les taches repetitives. Utiliser des scripts de dev. Configurer l'IDE pour tuer les processus zombies.

Difficulte 5 : Gestion des cles etrangeres lors des suppressions

Probleme : Suppression d'utilisateurs impossible si des emprunts/reservations existent (contrainte FOREIGN KEY).

Symptome : ERROR 1451 (23000): Cannot delete or update a parent row.

Solution : SET FOREIGN_KEY_CHECKS = 0 temporairement. Suppression en cascade des entites liees. Dans le code, ajout de @OnDelete(action = OnDeleteAction.CASCADE) sur les relations JPA.

Lecon : Bien modeliser les relations et les contraintes des le debut. Prevoir la strategie de suppression (CASCADE, SET NULL, RESTRICT) selon le metier.

7.2 Difficultes organisationnelles

- **Coordination equipe distante** : Daily Scrum parfois difficiles a organiser. Solution : Daily asynchrone via Slack + reunion bi-hebdomadaire.
- **Estimation des User Stories** : Premiere iteration avec surestimation. Amelioration apres Sprint 1 avec Planning Poker.
- **Gestion du Product Backlog** : Manque de refinement initial. Sprint 2 perturbe par changements priorites. Solution : Backlog Refinement mi-sprint instaure.

7.3 Lecons apprises cles

- La methodologie Scrum facilite la livraison incrementale et la gestion des priorites changeantes
- Les tests unitaires detectent rapidement les regressions et donnent confiance lors des refactorings
- Les Design Patterns ne sont pas academiques : ils resolvent des problemes reels et facilitent l'evolution du code
- La documentation de l'API (meme informelle) est essentielle pour la collaboration equipe
- Le versionnement Git avec messages descriptifs ameliore grandement la tracabilite et le debugging
- Commencer simple puis refactorer vaut mieux que surconcevoir des le debut
- Les revues de code (meme informelles) ameliorent la qualite et partagent les connaissances
- Automatiser les taches repetitives (tests, deploiement) libere du temps pour les fonctionnalites
- Les contraintes des plateformes cloud doivent etre anticipees des la conception
- La communication continue avec le Product Owner evite les malentendus et les refontes

8. Conclusion

8.1 Bilan du projet

Ce projet de fin de cours a permis de developper une application web complete de gestion de bibliotheque universitaire en appliquant rigoureusement les meilleures pratiques du Genie Logiciel et la methodologie Agile Scrum.

Les objectifs initiaux ont ete atteints et meme depasses :

Objectif	Cible	Realise	Statut
Application fonctionnelle	3 roles operationnels	3 roles + 10 fonctionnalites principales	■ Depasse
Tests unitaires	Couverture >= 65%	74.8% avec 47 tests	■ Depasse
Design Patterns	Minimum 4	4 patterns bien integres	■ Atteint
User Stories	15 US livrees	14 US Done, 1 US en cours	■ Quasi-atteint
Deploiement cloud	Application accessible en ligne	Deploiement Render operationnel	■ Atteint
Versionnement	Code sur GitHub	20+ commits, branches, PR	■ Depasse
Methodologie Scrum	3 sprints documentes	Product Backlog + ceremonies tracees	■ Atteint

8.2 Competences acquises

Sur le plan technique, le projet a permis de maitriser :

- Le framework Spring Boot et son ecosysteme (Security, Data, Web, Mail)
- La conception d'API REST securisees avec JWT
- L'ORM Hibernate/JPA et la modelisation relationnelle avancee
- Les tests unitaires avec JUnit, Mockito et JaCoCo
- Les Design Patterns en contexte reel de developpement
- Le deploiement d'applications containerisees Docker sur le cloud
- La gestion de bases de donnees MySQL en production

Sur le plan methodologique :

- L'application concrete de Scrum sur un projet de 6 semaines
- La redaction et l'estimation de User Stories selon les criteres INVEST
- L'utilisation d'outils de gestion de projet (GitHub Projects, Kanban)
- La tenue de ceremonies Scrum (Daily, Review, Retrospective)
- La collaboration en equipe sur un repository Git partage
- La redaction de documentation technique (rapport, conception, code)

8.3 Perspectives d'amelioration

Plusieurs pistes d'evolution ont ete identifiees pour enrichir le systeme :

Ameliorations fonctionnelles :

- Systeme de notation et commentaires des ouvrages par les etudiants
- Recommendations personnalisees basees sur l'historique d'emprunts
- Notifications SMS en plus des emails (Twilio API)
- Interface mobile native (React Native ou Flutter)

- Systeme de prolongation d'emprunt en ligne
- Gestion des collections numeriques (e-books, articles PDF)

Ameliorations techniques :

- Migration vers une architecture microservices pour la scalabilite
- Implementation d'un cache Redis pour les requetes frequentes
- Ajout d'une API GraphQL en complement du REST
- Tests d'integration et tests end-to-end avec Selenium
- Monitoring et alerting avec Prometheus + Grafana
- Pipeline CI/CD complet avec GitHub Actions

8.4 Conclusion personnelle

Ce projet a ete formateur a plusieurs niveaux. Il a permis de mettre en pratique l'ensemble des concepts vus en cours dans un contexte realiste avec des contraintes reelles (deadlines, travail d'equipe, integration continue). La methodologie Scrum s'est revelee particulierement adaptee pour gerer un projet de cette envergure avec des priorites evolutives.

Les difficultes rencontrees, notamment sur Spring Security et le deploiement cloud, ont ete autant d'opportunités d'apprentissage. Le debugging systematique, la lecture de documentation officielle et l'entraide au sein de l'equipe ont ete essentiels pour les surmonter.

L'application finale, bien que perfectible, repond aux besoins exprimes et respecte les exigences de qualite fixees. Elle constitue une base solide qui pourrait etre deployee en production avec quelques ajustements mineurs (monitoring, backups automatiques, gestion des erreurs renforcee).

Ce projet nous a prepares aux defis du developpement logiciel professionnel et nous a donne confiance dans notre capacite a mener a bien des projets complexes en appliquant les bonnes pratiques de l'industrie.